



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



“EJERCICIO 2, EL SISTEMA GESTOR EN UN CONTENEDOR”

EQUIPO:

DANAET MONTSERRATH GONZÁLEZ MARTÍNEZ

GARCIA CASTILLO MARIO

ISLAS RAMIREZ DIEGO

GRUPO: 3CV2

MATERIA: BASE DE DATOS

PROFESOR: GABRIEL HURTADO AVILÉS

El sistema gestor en un contenedor: Docker

Los contenedores son una forma más ligera y ágil de gestionar la virtualización, ya que no utilizan un hipervisor. Esto permite disfrutar de un aprovisionamiento de recursos más rápido y una disponibilidad más rápida de nuevas aplicaciones. La contenedorización agrupa todo lo necesario para ejecutar una sola aplicación o microservicio, junto con las bibliotecas de tiempo de ejecución que necesitan para funcionar: el código, sus dependencias e incluso el propio S.O. Esto permite que las aplicaciones se ejecuten prácticamente en cualquier lugar: una computadora de escritorio, una infraestructura de TI tradicional o la nube, aprovechando las características del S.O. anfitrión para aislar los procesos y controlan su acceso a las CPU, la memoria y el espacio de escritorio.

Al comparar los contenedores frente a las máquinas virtuales, la diferencia radica principalmente en tres aspectos:

	CONTENEDOR	MÁQUINA VIRTUAL
ARRANQUE	En segundos	En minutos
TAMAÑO	Megabytes	Gigabytes
AISLAMIENTO	Lo suficiente para que tres gestores convivan sin estorbarse	Mucho, pues levantar tres gestores significa tener tres sistemas operativos corriendo a la vez

Por otro lado, los componentes fundamentales para trabajar con contenedores son:

- Imagen: Es una plantilla ejecutable de solo lectura (por ejemplo, PostgreSQL 17 listo para usarse) que contiene el código, las dependencias y los archivos base del sistema requeridos.
- Contenedor: Es una instancia en ejecución de esa imagen. Funciona de manera aislada y es posible desplegar múltiples contenedores independientes partiendo de la misma imagen base.
- Volumen: Es el espacio donde viven los datos fuera del contenedor, permite guardar los datos en la máquina anfitriona para que no se pierdan al detener o eliminar el contenedor.
- Puerto Publicado: Es el mapeo que conecta un puerto de la máquina anfitriona con un puerto interno del contenedor. Permite acceder a los servicios expuestos desde

el exterior (por ejemplo, mapear el puerto local 5432 para PostgreSQL, 3306 para MySQL o 27017 para MongoDB).

Tenemos que tomar en cuenta que el volumen es indispensable porque los contenedores son efímeros; su almacenamiento interno es temporal y queda acoplado al ciclo de vida de la instancia. Si no se declara, la aplicación funcionará normalmente mientras el contenedor esté activo e incluso si se detiene, pero en el momento en que el contenedor se elimine todos los datos generados se perderán de forma permanente e irreversible, ya que su capa de lectura/escritura se destruye junto con él.

Fuentes Bibliográficas:

- **IBM.** (2024). *Contenedores vs. máquinas virtuales: ¿Cuál es la diferencia?* IBM Think. <https://www.ibm.com/mx-es/think/topics/containers-vs-vm>
- **Microsoft.** (2023, 23 de octubre). *Contenedores de Windows frente a máquinas virtuales.* Microsoft Learn. <https://learn.microsoft.com/es-es/virtualization/windowscontainers/about/containers-vs-vm>
- **Docker.** (s. f.). *Volumes.* Docker Docs. Recuperado el 15 de septiembre de 2026, de <https://docs.docker.com/engine/storage/volumes/>